You said:

Pasted text(20261030-070559).txt
Document

translate to english

ChatGPT said:

The uploaded transcript is in Hindi/Hinglish. Below is its English translation, preserving the original speaker's explanations, examples, technical terminology, and overall structure.

System Design – Application Architecture: Monolithic Architecture

1. Introduction

Hello everyone! Welcome to this new video.

You are currently following our System Design course, which is part of the Ultimate Placement Series.

In the first phase of System Design, we have already designed our first system: a **URL Shortener**.

Ideally, today I should have taught you the second system. However, since I am teaching this course on YouTube, I thought there are a few topics that are extremely important for today's software engineers and developers.

If you want to understand how modern applications are designed, these concepts are essential.

Before moving to the next system, where I will probably teach you how to design a Rate Limiter, I want to cover an additional topic that I had not originally planned for this course.

I believe this topic will be extremely helpful for you.

Today, we are going to discuss **different types of application architectures**.

Whenever you work for a company and build its digital presence, you will generally develop applications in one of two formats:

1. Web Applications
2. Mobile Applications

Whether you are building a web application or a mobile application, what architecture should you follow?

What architectural principles are used to design modern applications?

We will understand:

- What architectural options are available?
- What are their challenges?
- When should you use which architecture?
- What architectural decisions should you make?

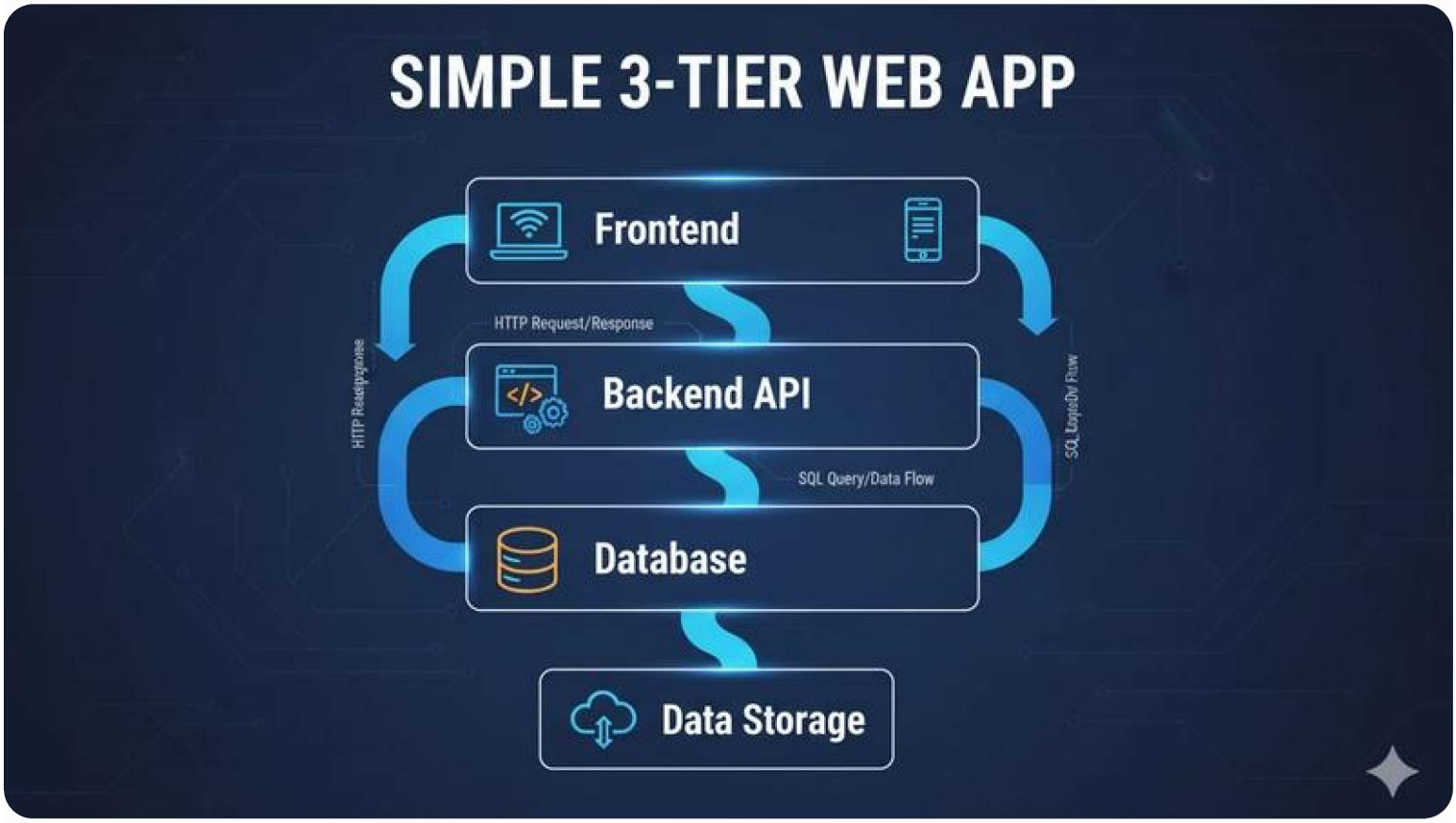
These topics are also very important from an interview perspective because interviewers can ask several questions about them.

By the end of this session, we will try to cover the major architectural decisions involved in designing modern web and mobile applications.

Let's get started!

2. Three Layers of an Application

Whenever you build an application, whether it is a mobile application or a web application, it generally consists of three layers.



Layer 1: Visualization Layer (Frontend / UI)

The first layer is called the **Visualization Layer**.

You can also call it:

- Frontend
- User Interface (UI)

What exactly is this layer?

Imagine an end user accessing your application through:

- A mobile phone
- A laptop browser

Whatever the end user sees and interacts with is called the Visualization Layer.

For example:

If I want to use Instagram and open it on my mobile phone, whatever appears on my screen is the frontend.

Similarly, if I open a browser on my laptop and type `www.linkedin.com`, the page that opens is the Visualization Layer.

This is the layer with which the end user directly interacts.

Layer 2: Backend (Brain of the System)

The second layer is called the **Backend**.

I also call it the Brain of the System.

This is where you implement all the application's business logic and data-processing operations.

This is where data processing and manipulation happen.

Consider Instagram.

When you open Instagram on your mobile phone, you see thousands of videos and reels.

Does all that content exist on your mobile phone?

No.

When your phone is connected to the internet, the backend continuously provides the required data to your application.

The backend is responsible for supplying this data.

Similarly:

- When you open LinkedIn in your browser, the backend sends the data to your browser.

- When you open Netflix or YouTube, you can access thousands or millions of videos.
- These videos are not stored on your laptop. You access them remotely over the internet through the backend.

Therefore, the backend is the brain of the system, where the application's logic and data processing happen.

Layer 3: Storage Layer

The third layer is called the **Storage Layer**.

This is where you store your data.

Every application, whether small, medium-sized, or extremely large, needs a storage layer to ensure that user data is not lost.

You save the application's data in this layer.

Therefore, every modern application generally consists of these three layers:

1. Visualization Layer – Frontend / UI
2. Backend Layer – Business Logic
3. Storage Layer – Database / Data Storage

How you implement and organize these three layers depends on the architectural decisions you make.

---

3. Monolithic Architecture

The first architectural approach we will discuss is called **Monolithic Architecture**.

The word Monolithic consists of two parts:

- Mono = One
- Lithic = Large structure / single mass

In simple terms, Monolithic Architecture refers to building an application as one large, unified system.

What is Monolithic Architecture?

In a conventional monolithic application, the three components are developed together:

- Frontend
- Backend
- Storage Layer

The traditional definition presented in this session is based on two conditions:

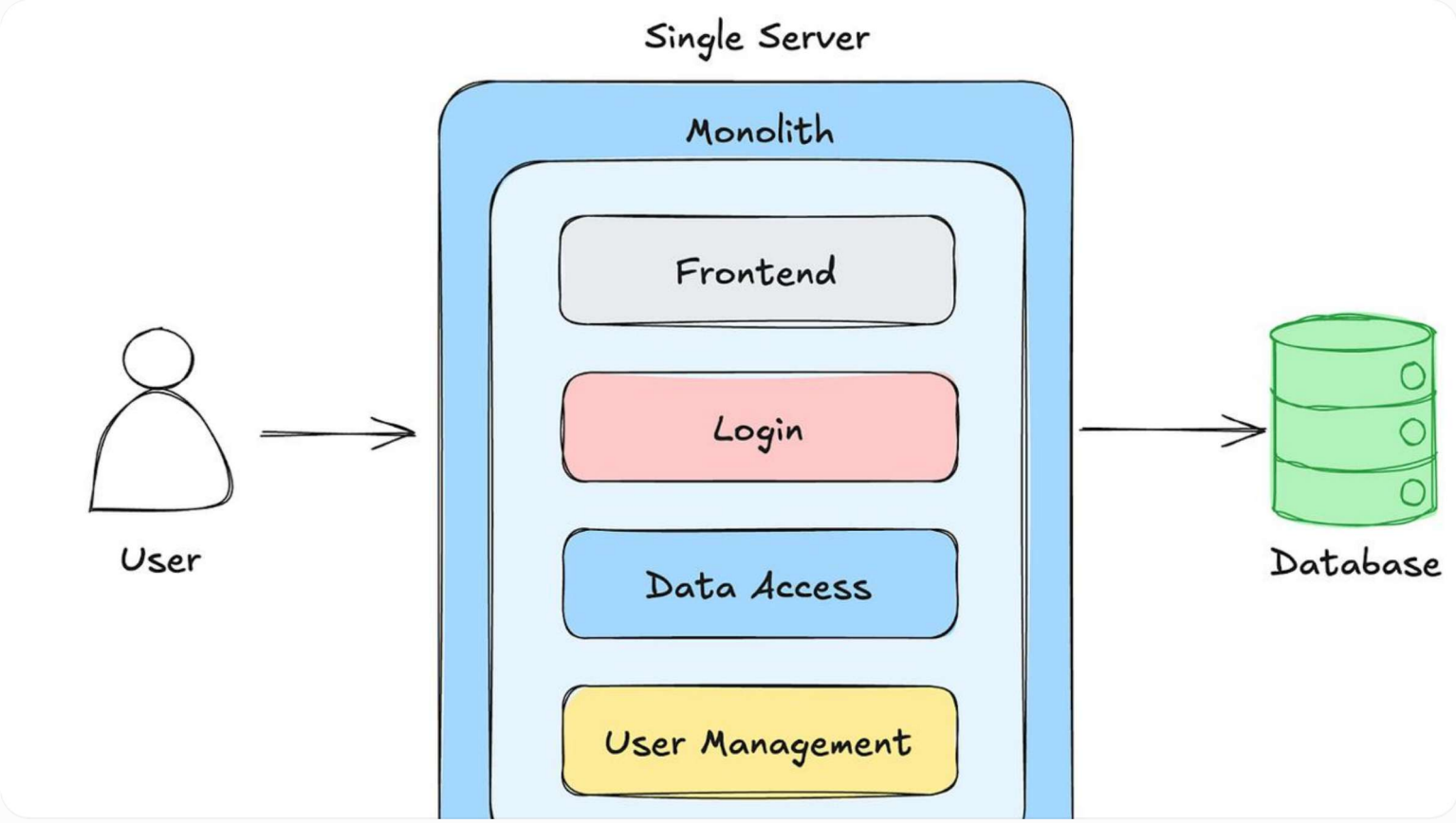
1. All components are maintained within a single codebase.
2. All components are deployed together on the same server/system.

If these conditions are satisfied, we call it a Monolithic Architecture.

In simple terms:

You write all the application code in one repository or codebase and deploy the application as one unit.

Monolithic Architecture Diagram



Imagine that you have one server.

Inside that server, you have:

- Frontend
- Backend
- Database / Storage Layer

Everything is deployed together.

The code structure also contains all three components within the same application codebase.

This is what we call a Monolithic System.

---

4. When and Why Should You Use Monolithic Architecture?

The next important question is:

When should you use Monolithic Architecture, and why?

There are several situations where a monolithic application makes sense.

Reason 1: When You Are Just Starting

Use Monolithic Architecture when you are starting a new application.

For example:

- You do not know how the application will evolve in the future.
- You are unsure about future scalability requirements.
- You do not know how complex the application will become.
- You want to build a Proof of Concept (POC).
- You want to develop the application quickly.

In such situations, starting with a monolithic application makes sense.

You do not need to introduce unnecessary architectural complexity from day one.

Reason 2: When You Need Very Low Latency

If you are building an application that is extremely latency-sensitive and requires very fast response times, Monolithic Architecture can make sense.

Why?

Because the frontend, backend, and database components are deployed within the same system.

According to the speaker, communication between these components becomes faster because network communication between separately deployed components is reduced.

This can help achieve faster response times.

The speaker gives an example from his experience working at LinkedIn.

They needed to build an application that was extremely latency-sensitive.

They went back to the traditional approach of doing things, using a monolithic design, to achieve the required latency.

Therefore, if you need very low latency, a monolithic architecture may be a suitable option.

Reason 3: When You Need High Security

Another situation where Monolithic Architecture can make sense is when you want to build a highly secure application without introducing unnecessary security complexity and costs.

Let me explain this with an example.

Imagine that your country has only one Prime Minister.

If you need to protect that Prime Minister, you can deploy several security guards around that single person.

All the security guards focus on protecting one individual.

Now imagine another situation.

Suppose your country has multiple Prime Ministers:

- PM 1
- PM 2
- PM 3

All three are equally important and need protection.

However, your security resources are limited.

For example, you have only five security guards.

You might deploy:

- Two guards around PM 1
- Two guards around PM 2
- One guard around PM 3

Because your security resources are distributed across multiple locations, each Prime Minister receives less protection.

The possibility of an attack increases.

The speaker uses this analogy to explain that when you have one centralized system, it can be easier and less expensive to secure because security resources and controls are concentrated in one place.

Therefore, if you need to build a highly secure system, Monolithic Architecture may be a suitable choice.

The speaker also mentions banking systems and other highly secure applications as examples.

Reason 4: Easier Testing

Another advantage of Monolithic Architecture is that testing becomes easier.

Why?

Because everything is maintained together:

- One application
- One codebase
- One deployment unit

This can simplify testing and coordination between components.

Reason 5: Lower Cost

Monolithic Architecture can also be less expensive.

Why?

Because multiple components run together on the same machine or deployment environment.

You do not necessarily need to maintain multiple independently deployed services and their associated infrastructure.

The speaker gives an example of Amazon Prime.

He mentions that when Amazon Prime moved from a microservices architecture back to a monolithic architecture, its infrastructure costs were reduced significantly, reportedly by at least 10 times.

The main point is that running a simpler, unified application can reduce infrastructure and operational costs.

Speaker's Recommendation

Whenever you are building a new application and:

- You do not know how much it will scale.
- You do not know how complicated it will become.
- You are building a POC.
- You want to keep costs low.
- You need a simple architecture.

You should consider starting with a Monolithic Architecture.

There is nothing wrong with building a monolithic application, even in today's modern technology landscape.

The speaker mentions companies such as LinkedIn, Meta, and PayPal as examples of companies that started with monolithic systems and later moved towards service-oriented and microservices architectures.

5. Challenges of Monolithic Architecture

Everything is not perfect with Monolithic Architecture.

If it had only advantages, why would we need other architectural approaches?

Let's understand its challenges.

Challenge 1: Technology and Programming Language Constraints

Because everything is maintained in the same codebase, you may be forced to use the same technology stack across the application.

Ideally, you might want to use:

- One language for the frontend.
- Another language for the backend.
- Another technology for data processing.

Different technologies might be more suitable for different components.

However, in a tightly coupled monolithic implementation, maintaining everything together can introduce technology constraints.

The speaker gives Facebook as an example.

Initially, Facebook developed its application using PHP.

Both frontend and backend development were closely associated with PHP.

Today, using the same technology for every component may not always be the most optimized approach.

Therefore, one challenge is **technology and programming language constraints**.

Challenge 2: Deployment Takes More Time

The second challenge is deployment.

Because everything belongs to the same codebase and deployment unit, even a small change may require redeploying the entire application.

For example:

Suppose you make a small CSS change in the frontend.

You only changed the color of a button.

Ideally, you would want to deploy only the frontend change.

However, in the monolithic approach described here, you may need to redeploy:

- Frontend
- Backend
- Storage-related application components

Therefore, even a small change can require deploying the entire application.

This makes deployment slower and potentially more complicated.

Challenge 3: Selective Scaling Is Not Possible

This is one of the biggest challenges.

Does that mean a monolithic application cannot scale?

No.

A monolithic application can scale.

You can scale it:

- Vertically
- Horizontally

However, it has its own scalability challenges.

Let's understand this with an example.

Imagine that we have a monolithic application running on one server.

Inside that server, we have:

- Frontend
- Backend
- Database

Now imagine that this application is the Times of India website.

Initial Situation

When the website was initially launched, only a small number of people visited it.

One server was sufficient to handle the traffic.

Over time, the website started publishing more engaging content, images, and other news.

As a result, more and more people started visiting and reading the website.

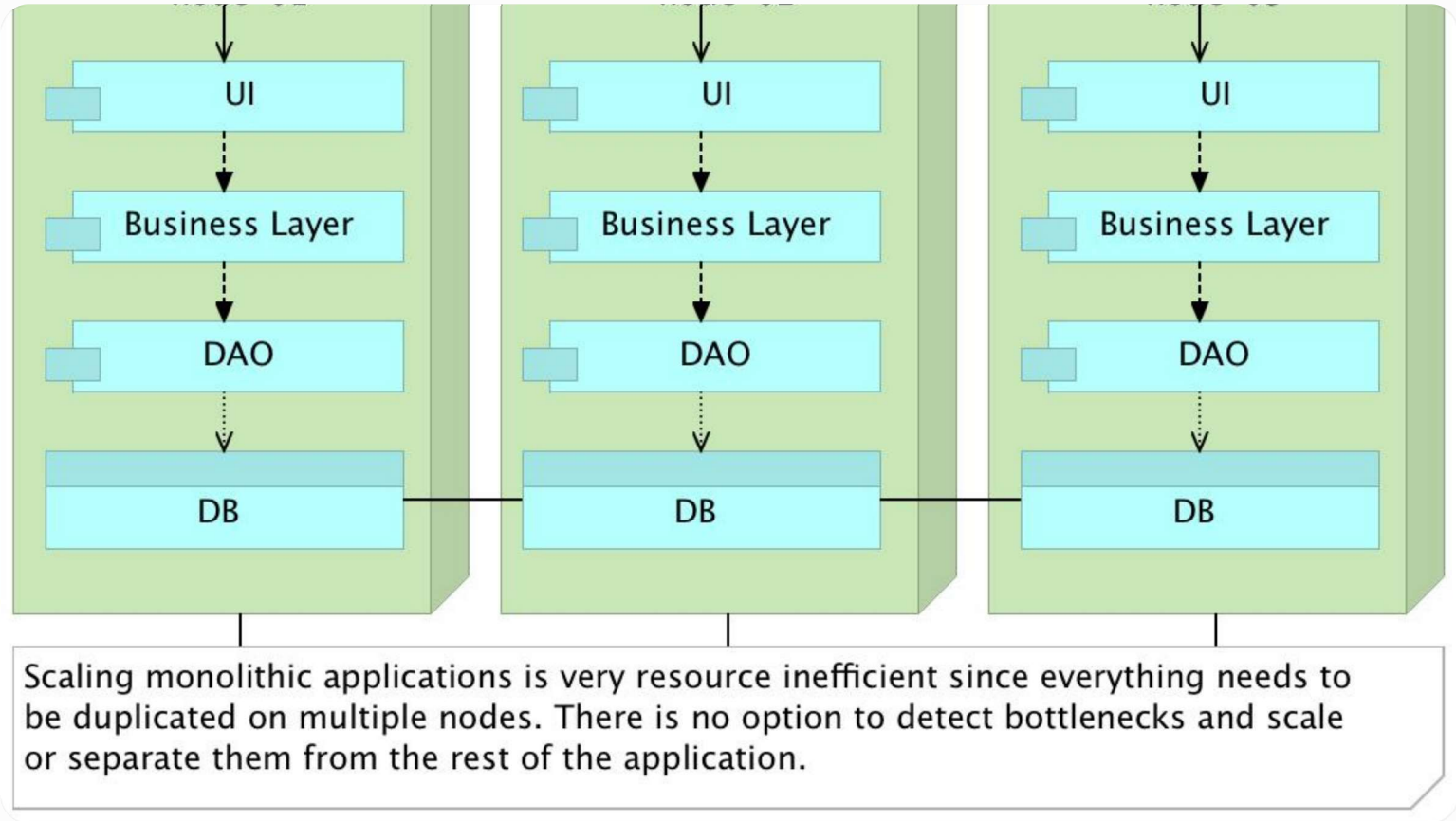
Now the application started receiving increased traffic.

We needed to scale the application.

Horizontal Scaling

Instead of running everything on one machine, we introduced multiple machines.

We horizontally scaled the monolithic application.



Now we have multiple machines, and each machine contains:

- Frontend
- Backend
- Database

We successfully increased the application's capacity.

However, there is a problem.

Understanding the Scaling Problem

Consider that Times of India is primarily a read-oriented application.

Users mostly read news articles.

They do not frequently write or modify content.

Therefore, when traffic increases, the load primarily increases on the frontend and read-serving components.

The backend and database may not experience the same proportional increase in load.

Ideally, we should scale only the frontend or the components experiencing increased traffic.

We should not unnecessarily scale the backend and database if they do not need additional resources.

However, because the application is monolithic, scaling the application means scaling all its components together.

Even if you only need additional frontend capacity, you end up scaling:

- Frontend
- Backend
- Database

This increases infrastructure costs and operational complexity.

The speaker explains this using the Hindi expression:

"When the flour is ground, the insects mixed with it are also ground."

In other words, components that do not need additional resources are also scaled unnecessarily.

Therefore, the challenge is:

You can scale a monolithic application, but selective scaling is difficult because all components are scaled together.

Challenge 4: Human Dependency and Reduced Development Speed

Consider the following situation.

- Sheetal works on the frontend.
- Rahul works on the backend.

Both developers are working on the same codebase.

Now suppose Rahul wants to make a change in the application.

He may need to coordinate with Sheetal.

Similarly, if Sheetal wants to make a change, she may need to coordinate with Rahul.

Why?

Because everyone is working within the same codebase, and changes can affect shared components.

This creates significant human dependency.

As a result:

- Developers become dependent on each other.
- Coordination takes more time.
- Development becomes slower.
- Team agility decreases.
- Overall development speed decreases.

This is another challenge of Monolithic Architecture.

6. Quick Revision: Monolithic Architecture

Let's quickly summarize everything we have learned.

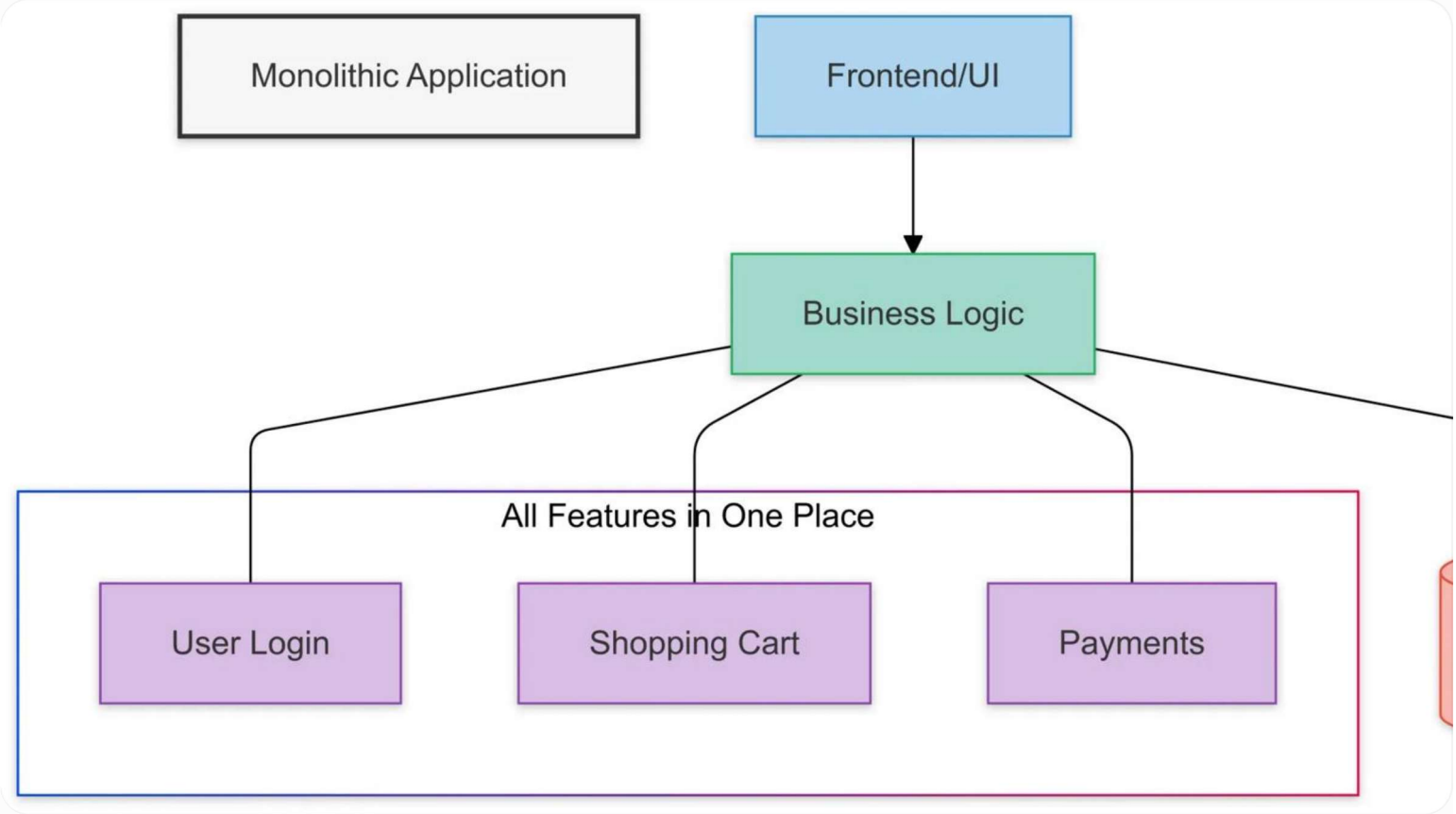
Application Architecture: Three Layers

Every application generally consists of three layers:

Layer	Responsibility
Frontend / Visualization Layer	User interface that the end user interacts with
Backend Layer	Business logic, data processing, and manipulation
Storage Layer	Stores application and user data

What Is Monolithic Architecture?

When the frontend, backend, and storage-related application components are maintained in a unified codebase and deployed together as one application, it is commonly called Monolithic Architecture.



Advantages of Monolithic Architecture

Advantage	Explanation
Simple to start	Suitable for new applications and POCs
Low latency potential	Fewer network boundaries between components
Security	Centralized deployment can simplify security controls
Easier testing	Unified application can simplify testing
Lower cost	Fewer independently managed services and infrastructure components

Challenges of Monolithic Architecture

Challenge	Explanation
Technology constraints	Difficult to independently choose technologies for tightly coupled components
Deployment complexity	A small change may require redeploying the entire application
No selective scaling	Components may need to scale together, even when only one component needs additional capacity
Low agility	Developers may depend on each other because they share the same codebase

7. What Comes Next?

We have now understood:

1. What application architecture means.
2. The three layers of an application.
3. What Monolithic Architecture is.
4. When and why we should use Monolithic Architecture.
5. Its advantages.
6. Its challenges.

The next step is to understand why we moved towards other architectural approaches.

Because of the challenges associated with monolithic applications, we started exploring other architectural models.

In the next video, we will discuss:

- Service-Oriented Architecture (SOA)
- Microservices Architecture

The speaker concludes the session by asking viewers to confirm whether they have understood Monolithic Architecture and its advantages and disadvantages.

Important interview takeaways

These are the key questions arising from the session:

1. What is Monolithic Architecture?
2. What are the three layers of a typical application?
3. When should you choose Monolithic Architecture?
4. What are the advantages of Monolithic Architecture?
5. What are the challenges of Monolithic Architecture?
6. Can a monolithic application scale horizontally?
7. What is selective scaling, and why is it difficult in a monolithic system?
8. Why can deployment become slower in a monolithic application?
9. How does a monolithic architecture affect developer collaboration?
10. Why might a company start with a monolith and later migrate towards microservices?

Note: This is a translation of the speaker's explanation, not an independent technical validation. Some statements, particularly that monoliths must use one programming language, that their network I/O is zero, and that they are inherently more secure, are simplified explanations rather than universal architectural rules.

A monolith can use multiple technologies, separate database infrastructure, and multiple servers. These distinctions are important for system design interviews.